

COVID-19_Data_Report_CA(2020-2023)

Nalini Thangavelu

2026-10-01

Aim of the Report

This report imports, tidies and analyzes the COVID-19 data set from the *John Hopkins GitHub* site to evaluate and compare monthly infection trends across five selected California counties (Alameda, Santa Clara, Santa Cruz, Los Angeles, and San Francisco) from January 2020 to March 2023.

Source URLs: 'https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_confirmed_US.csv' 'https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_deaths_US.csv'

The above dataset provides information on the confirmed cases and deaths, reported at the county level for US from January 2020 to March 2023. All data was read in from the daily case report.

Required Packages

Please install the below list of packages that are used in this analysis.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.3      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
# Required to clean_names of the data frame
library(janitor)
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##   chisq.test, fisher.test
```

Import the data

Import the raw data from the source url

```
main_url <- paste0(
  "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/",
  "master/csse_covid_19_data/csse_covid_19_time_series/"
)
covid_dataset <- c("confirmed_US", "deaths_US")
# read the data from the individual csv files and assign it to the variables
# named from covid_dataset vector
for (i in 1:length(covid_dataset)){
  file_url <- paste(main_url,"time_series_covid19_",covid_dataset[i],".csv",sep="")
  assign(covid_dataset[i],read.csv(file_url))
}
print(paste("Data read from CSV files are in: ", toString(covid_dataset)))
```

```
## [1] "Data read from CSV files are in: confirmed_US, deaths_US"
```

Tidy the data

Clean and transform the data using the functions from the package tidyverse.

For each of the US data frame imported from the corresponding csv file, try to

1. Convert all the columns corresponding to the observed dates as rows using `pivot_longer` function.
2. Remove the columns that we are not interested in for this analysis with `select`
3. Change all the column names to follow `snake_case` for easier readability
4. Rename *Admin2* field to *county* for better clarity
5. Correct the date value in this character format `X%m.%d.%y` (e.g. value `X1.22.20`) to period data type

Clean the Data

```
cases_usa <- confirmed_US |>
  pivot_longer(
    cols=c("UID", "iso2", "iso3", "code3", "FIPS", "Admin2",
           "Province_State", "Country_Region", "Lat", "Long_",
           "Combined_Key"),
    names_to="date",
    values_to="cases"
  ) |>
  dplyr::select(
    cols=c("UID", "Lat", "Long_", "iso2", "iso3", "code3", "FIPS")
  ) |>
  rename(county = Admin2) |>
  janitor::clean_names()
cases_usa$date <- as.Date(cases_usa$date,format="X%m.%d.%y")

deaths_usa <- deaths_US |>
  pivot_longer(
    cols=c("UID", "iso2", "iso3", "code3", "FIPS", "Admin2",
```

```

        "Province_State", "Country_Region", "Lat", "Long_",
        "Combined_Key", "Population"),
    names_to="date",
    values_to="deaths"
) |>
dplyr::select(
  cols=c("UID", "Lat", "Long_", "iso2", "iso3", "code3", "FIPS")
) |>
rename(county = Admin2) |>
janitor::clean_names()
deaths_usa$date <- as.Date(deaths_usa$date,format="%m.%d.%y")

```

Transform the Data

```

# Combine both cases_usa and deaths_usa to create one single data frame to have
# the population information.
covid_usa <- cases_usa |>
  full_join(deaths_usa)

```

```

## Joining with 'by = join_by(county, province_state, country_region,
## combined_key, date)'

```

```

print(glimpse(covid_usa))

# Take the covid_usa data and from that get the number of cases monthly over
# the data collection period for the various counties in California State
covid_ca_monthly <- covid_usa |>
  # filter out all the records for California State
  filter(`province_state` == 'California') |>
  # Group by county, then end of the month
  group_by(county, monthly_date = ceiling_date(date, "month") - days(1)) |>
  # Since the cases is a cumulative count, lets get the max count for the month
  summarise(
    cases = max(cases, na.rm = TRUE),
    population = max(population),
    .groups = "drop"
  )

print(glimpse(covid_ca_monthly))

```

Visualize and Analyze the Data

With the transformed data, visualize the monthly trend of confirmed COVID-19 cases for these five California counties: Alameda, Santa Clara, Santa Cruz, San Francisco and Los Angeles.

```

# Create chart_data by filtering the selected counties from the covid_ca_monthly
chart_data <- covid_ca_monthly |>
  # Filter for only the selected counties
  filter(
    county %in% c('Alameda', 'Santa Clara', 'Santa Cruz',

```

```

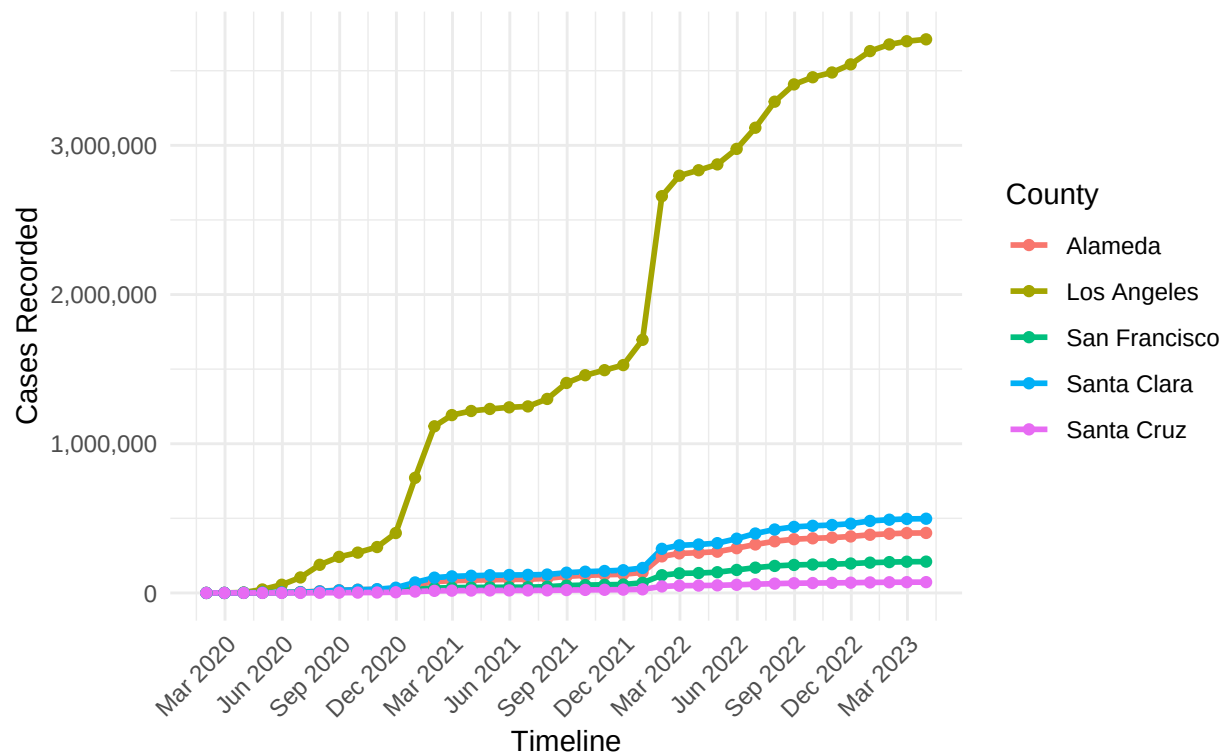
    'San Francisco', 'Los Angeles')
  ) |>
  rename(date_axis = monthly_date)

# Build a line chart for comparison
ggplot(chart_data, aes(x = date_axis, y = cases, color = county, group = county)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.5) +
  scale_x_date(date_breaks = "3 months", date_labels = "%b %Y") +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "COVID-19 Trend Across California Counties (Cumulative counts)",
    subtitle = paste0("Comparing Alameda, Santa Clara, Santa Cruz, Los Angeles",
                      "and San Francisco counties"),
    x = "Timeline",
    y = "Cases Recorded",
    color = "County"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

COVID-19 Trend Across California Counties (Cumulative counts)

Comparing Alameda, Santa Clara, Santa Cruz, Los Angeles and San Francisco counties



```

# Transform the data further to visualize the trend based on the new cases for the month
covid_ca_monthly_discrete <- covid_ca_monthly |>
  group_by(county) |>
  arrange(monthly_date, .by_group = TRUE) |>

```

```

mutate(cases = cases - lag(cases, default=0))

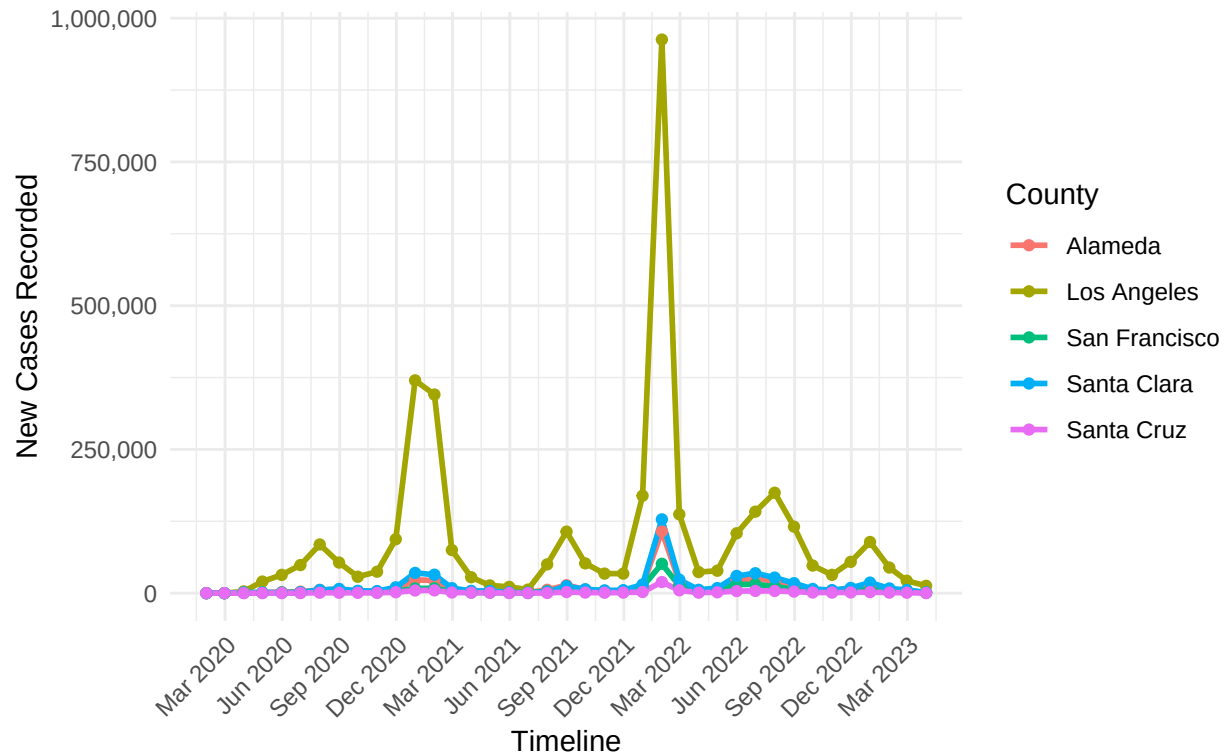
chart_data2 <- covid_ca_monthly_discrete |>
  # Filter for only the selected counties
  filter(
    county %in% c('Alameda', 'Santa Clara', 'Santa Cruz', 'San Francisco',
                  'Los Angeles')
  ) |>
  rename(date_axis = monthly_date)

ggplot(chart_data2, aes(x = date_axis, y = cases, color = county, group = county)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.5) +
  scale_x_date(date_breaks = "3 months", date_labels = "%b %Y") +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "COVID-19 Trend Across California Counties",
    subtitle = paste0(
      "Comparing Alameda, Santa Clara, Santa Cruz,",
      "Los Angeles and San Francisco counties"
    ),
    x = "Timeline",
    y = "New Cases Recorded",
    color = "County"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

COVID-19 Trend Across California Counties

Comparing Alameda, Santa Clara, Santa Cruz, Los Angeles and San Francisco counties

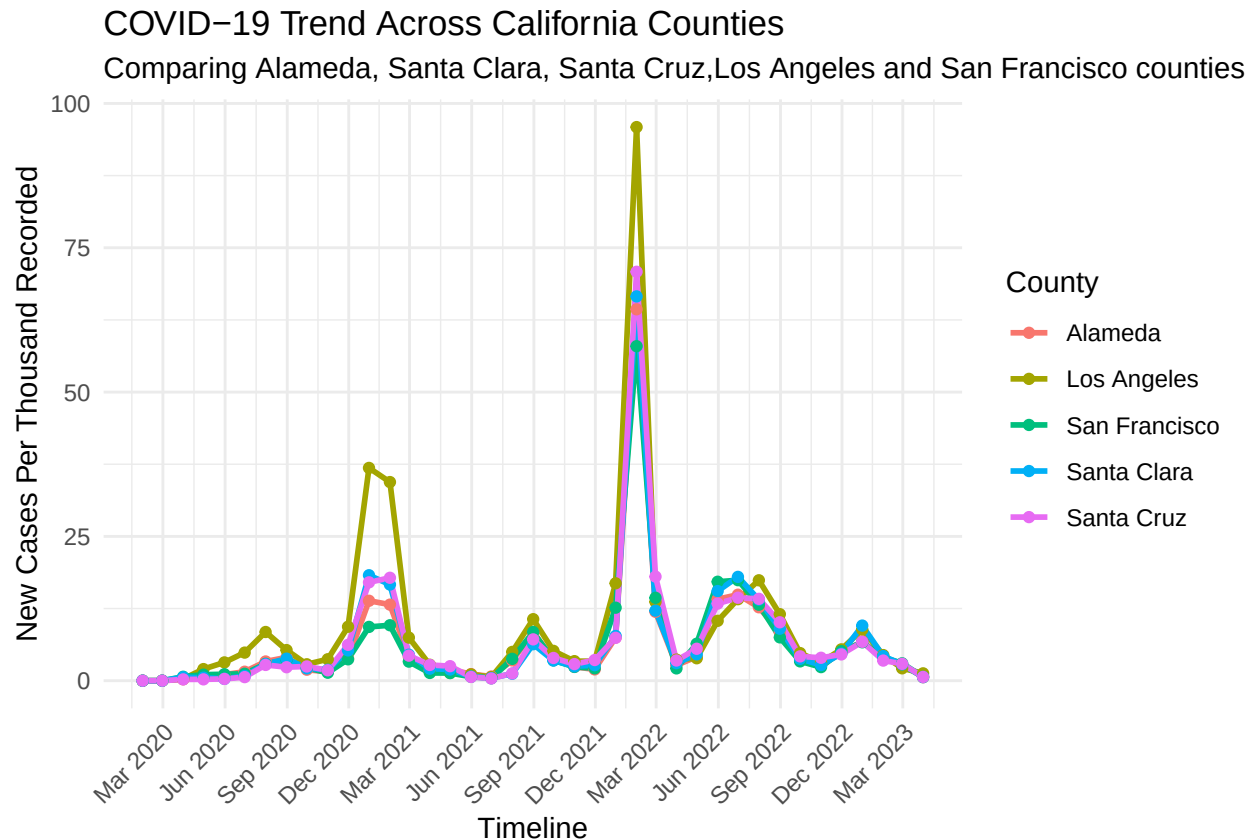


```
# Tie in the population info to visualize the trend of new cases per thousand
covid_ca_per_thousand <- covid_ca_monthly_discrete |>
  mutate(cases_per_thousand = cases * 1000 / population )

chart_data3 <- covid_ca_per_thousand |>
  # Filter for only the selected counties
  filter(
    county %in% c('Alameda', 'Santa Clara', 'Santa Cruz', 'San Francisco',
                  'Los Angeles')
  ) |>
  rename(date_axis = monthly_date)

ggplot(chart_data3, aes(x = date_axis, y = cases_per_thousand, color = county, group = county)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.5) +
  scale_x_date(date_breaks = "3 months", date_labels = "%b %Y") +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "COVID-19 Trend Across California Counties",
    subtitle = paste0(
      "Comparing Alameda, Santa Clara, Santa Cruz,",
      "Los Angeles and San Francisco counties"
    ),
    x = "Timeline",
    y = "New Cases Per Thousand Recorded",
    color = "County"
  )
```

```
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



From the above charts, we can see that number of confirmed COVID-19 cases has peaked in the selected California counties around the winters of 2020 and 2021. These correctly correspond to the Winter 2020–2021 surge and the Winter 2021–2022 Omicron surge seen in these counties. The summer surge of 2022 corresponds to a wave of infections driven by the highly transmissible omicron subvariant BA.5.

Model the Data

Let us see how we can model this data to predict the number of new cases based on the county, population and monthly_date. Since we are making predictions on the new infections monthly, we will go with Generalized Linear Model with log link function and $\log(\text{population})$ offset to account for the varying county sizes. Initial testing under a standard Poisson family revealed extreme over-dispersion, meaning the variance vastly exceeded the mean due to the clustered, wave-like nature of epidemic spikes. To resolve this violation of assumptions, had to transition to a Quasi-Poisson GLM. This model dynamically estimated a realistic dispersion parameter of 10973.99, widening our standard errors to true historical scales and ensuring our p-values and subsequent prediction boundaries are statistically valid.

```
# Let's try to model this data to predict the number of new cases based on
# the county, population and monthly_date.
model_data <- covid_ca_monthly_discrete |>
  filter(
    county %in% c('Alameda', 'Santa Clara', 'Santa Cruz', 'San Francisco',
```

```

        'Los Angeles')
    ) |>
    mutate(
      year = as.factor(year(monthly_date)),
      month = as.factor(month(monthly_date)),
      # To account for the huge difference in the sizes of counties as seen with
      # Santa Cruz to Los Angeles use log(population), force the model to look at
      # the per capita infection rate instead of just raw case counts
      log_pop = log(population) # Pre-calculating log of population for the offset
    )

# In the Johns Hopkins COVID-19 GitHub repository, cumulative total_cases
# sometimes underwent retroactive data corrections by local health departments.
# When a county over-reported cases one week and corrected it the next,
# calculating monthly new_cases (subtracting one month's total from the next)
# can result in a negative number. Let's make sure we don't have these negative
# counts for our selected counties.
model_data |> filter(cases < 0) |> dplyr::select(county, month, year, cases)

```

```

## # A tibble: 0 x 4
## # Groups:   county [0]
## # i 4 variables: county <chr>, month <fct>, year <fct>, cases <int>

```

```

# Initially tried with poisson model
poisson_model <- glm(
  cases ~ county + year + month + offset(log_pop),
  data = model_data,
  # Since we are looking at new cases, our counts can't be negative. A log link
  # function keeps the model mathematically grounded by ensuring all predicted
  # values are strictly greater than or equal to zero
  family = poisson(link = "log")
)

summary(poisson_model)

```

```

##
## Call:
## glm(formula = cases ~ county + year + month + offset(log_pop),
##      family = poisson(link = "log"), data = model_data)
##
## Coefficients:
##              Estimate Std. Error  z value Pr(>|z|)
## (Intercept)   -4.147045    0.001936 -2142.105 < 2e-16 ***
## countyLos Angeles    0.429226    0.001660   258.548 < 2e-16 ***
## countySan Francisco -0.010782    0.002693   -4.004 6.23e-05 ***
## countySanta Clara    0.070010    0.002120   33.016 < 2e-16 ***
## countySanta Cruz     0.096866    0.004037   23.997 < 2e-16 ***
## year2021           0.226495    0.001391  162.804 < 2e-16 ***
## year2022           1.061563    0.001204  881.774 < 2e-16 ***
## year2023          -1.253230    0.003173 -394.913 < 2e-16 ***
## month2            -1.672480    0.001903 -878.667 < 2e-16 ***
## month3            -2.789790    0.003146 -886.821 < 2e-16 ***

```

```
## month4          -2.733060    0.003146  -868.778 < 2e-16 ***
## month5          -1.987599    0.002238  -887.959 < 2e-16 ***
## month6          -1.763558    0.002031  -868.337 < 2e-16 ***
## month7          -1.420872    0.001761  -806.793 < 2e-16 ***
## month8          -1.497321    0.001817  -824.187 < 2e-16 ***
## month9          -2.257471    0.002525  -893.932 < 2e-16 ***
## month10         -2.501449    0.002824  -885.830 < 2e-16 ***
## month11         -1.939285    0.002191  -884.947 < 2e-16 ***
## month12         -0.754492    0.001373  -549.704 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
## Null deviance: 7461582 on 194 degrees of freedom
## Residual deviance: 2144802 on 176 degrees of freedom
## AIC: 2146783
##
## Number of Fisher Scoring iterations: 6
```

```
# Calculate the dispersion ratio
chi_sq <- sum(residuals(poisson_model, type = "pearson")^2)
deg_freedom <- poisson_model$df.residual
dispersion_ratio <- chi_sq / deg_freedom
print(paste("Dispersion Ratio: ", dispersion_ratio))
```

```
## [1] "Dispersion Ratio: 10973.9911492771"
```

```
# A standard Poisson model assumes that the variance in our data is exactly
# equal to the average case count. However, a pandemic does not spread evenly or
# randomly; it moves in massive, unpredictable waves. This causes overdispersion
# (dispersion ratio ~ 11000), meaning the actual swings in data are thousands of
# times larger than what a standard Poisson model expects.
```

```
# So let's switch to the Quasi-Poisson family for our model
quasi_poisson_model <- glm(
  cases ~ county + year + month + offset(log_pop),
  data = model_data,
  family = quasipoisson(link = "log")
)

summary(quasi_poisson_model)
```

```
##
## Call:
## glm(formula = cases ~ county + year + month + offset(log_pop),
##      family = quasipoisson(link = "log"), data = model_data)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -4.14705    0.20281  -20.448 < 2e-16 ***
## countyLos Angeles    0.42923    0.17391   2.468 0.014540 *
## countySan Francisco -0.01078    0.28210  -0.038 0.969555
```

```
## countySanta Clara      0.07001    0.22214    0.315 0.753007
## countySanta Cruz       0.09687    0.42285    0.229 0.819075
## year2021                0.22649    0.14574    1.554 0.121953
## year2022                1.06156    0.12612    8.417 1.30e-14 ***
## year2023               -1.25323    0.33244   -3.770 0.000223 ***
## month2                 -1.67248    0.19940   -8.388 1.56e-14 ***
## month3                 -2.78979    0.32955   -8.466 9.71e-15 ***
## month4                 -2.73306    0.32955   -8.293 2.76e-14 ***
## month5                 -1.98760    0.23449   -8.476 9.08e-15 ***
## month6                 -1.76356    0.21276   -8.289 2.83e-14 ***
## month7                 -1.42087    0.18449   -7.702 9.35e-13 ***
## month8                 -1.49732    0.19031   -7.868 3.52e-13 ***
## month9                 -2.25747    0.26455   -8.533 6.41e-15 ***
## month10                -2.50145    0.29582   -8.456 1.03e-14 ***
## month11                -1.93929    0.22957   -8.448 1.08e-14 ***
## month12                -0.75449    0.14378   -5.247 4.40e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 10973.99)
##
## Null deviance: 7461582 on 194 degrees of freedom
## Residual deviance: 2144802 on 176 degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 6
```

*# Note the dispersion parameter of 10973.99 in above output, which indicates our
data's swings to be roughly 11,000 times larger than a Poisson baseline.
Quasi-poisson family uses this number to widen the error margins, thereby
making the final p-values realistic*

```
eval_data <- model_data |>
  ungroup() |>
  mutate(predicted_cases = predict(quasi_poisson_model, type = "response"))

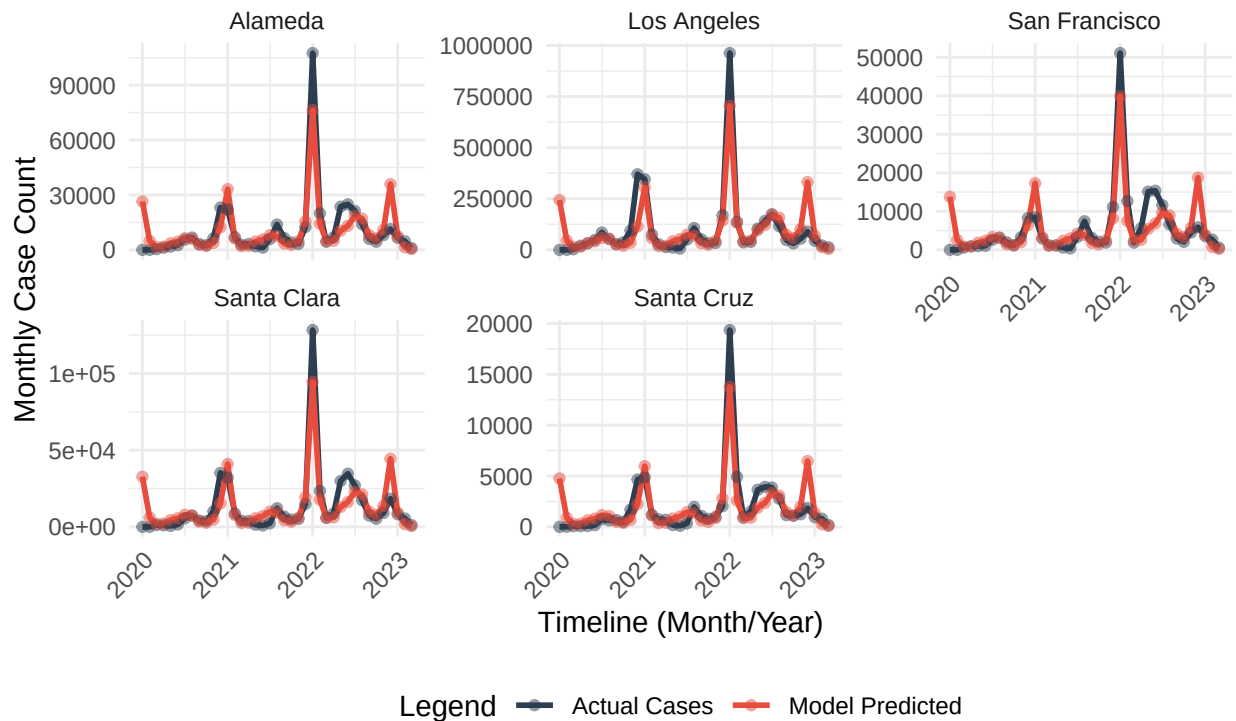
plot_data <- eval_data |>
  dplyr::select(county, year, month, cases, predicted_cases) |>
  # Combine year and month into a single date string for sequential
  # timeline plotting
  mutate(timeline = as.Date(paste(year, month, "01", sep = "-"), format = "%Y-%m-%d")) |>
  pivot_longer(
    cols = c(cases, predicted_cases),
    names_to = "Data_Type",
    values_to = "Count"
  ) |>
  mutate(
    Data_Type = recode(Data_Type,
      "cases" = "Actual Cases",
      "predicted_cases" = "Model Predicted")
  )

ggplot(plot_data, aes(x = timeline, y = Count, color = Data_Type, group = Data_Type)) +
  geom_line(linewidth = 1) +
```

```
geom_point(alpha = 0.5) +
# Use facet_wrap to allow each county to scale to its own population size
facet_wrap(~ county, scales = "free_y") +
labs(
  title = "COVID-19 Model Evaluation: Actual vs. Predicted Cases",
  subtitle = "Negative Binomial GLM with Population Offset",
  x = "Timeline (Month/Year)",
  y = "Monthly Case Count",
  color = "Legend"
) +
theme_minimal() +
scale_color_manual(values = c("Actual Cases" = "#2c3e50",
                              "Model Predicted" = "#e74c3c")) +
theme(legend.position = "bottom",
      axis.text.x = element_text(angle = 45, hjust = 1))
```

COVID-19 Model Evaluation: Actual vs. Predicted Cases

Negative Binomial GLM with Population Offset



Conclusion

Visualization and modeling reveal that per capita COVID-19 infection trends followed highly synchronized temporal patterns across the five analyzed California counties between January 2020 and March 2023. The data captures two dominant historical peaks: the *Winter 2020–2021* surge and the *Winter 2021–2022 Omicron* surge. Incorporating county as a fixed predictor alongside a population offset allowed a Quasi-Poisson GLM to capture regional baselines effectively, though the model slightly underestimated the magnitude of the 2021 winter surge peak.

A primary limitation of this analysis is selection bias; four of the five chosen counties reside in the San Francisco Bay Area, sharing highly correlated public health policies, regional demographics, and vaccination adoption patterns. Consequently, the model's high predictive performance may not generalize well to rural or conservative regions of California where intervention timelines differed. Furthermore, the underlying data suffers from severe reporting bias: early 2020 figures under-represent true cases due to strict PCR testing scarcity, while late 2022–2023 figures are structurally under-reported due to the widespread adoption of unrecorded at-home antigen tests.

AI Usage Statement

The Gemini 3.5 model developed by Google was utilized to brainstorm statistical modeling options, ultimately guiding the selection of the included Quasi-Poisson GLM.

Session Info

```
## R version 4.6.1 (2026-06-24)
## Platform: aarch64-apple-darwin23
## Running under: macOS Sonoma 14.6
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK version
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/Los_Angeles
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] janitor_2.2.1    lubridate_1.9.5 forcats_1.0.1  stringr_1.6.0
## [5] dplyr_1.2.1      purrr_1.2.2     readr_2.2.0   tidyr_1.3.2
## [9] tibble_3.3.1     ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] gtable_0.3.6      compiler_4.6.1    tidymodels_1.2.1  snakecase_0.11.1
## [5] scales_1.4.0      yaml_2.3.12       fastmap_1.2.0     R6_2.6.1
## [9] labeling_0.4.3    generics_0.1.4    knitr_1.51        pillar_1.11.1
## [13] RColorBrewer_1.1-3 tzdb_0.5.0        rlang_1.3.0       utf8_1.2.6
## [17] stringi_1.8.9     xfun_0.60         S7_0.2.2          otel_0.2.0
## [21] timechange_0.4.0  cli_3.6.6         withr_3.0.3       magrittr_2.0.5
## [25] digest_0.6.39     grid_4.6.1        rstudioapi_0.19.0 hms_1.1.4
## [29] lifecycle_1.0.5   vctrs_0.7.3       evaluate_1.0.5    glue_1.8.1
## [33] farver_2.1.2      rmarkdown_2.31    tools_4.6.1       pkgconfig_2.0.3
## [37] htmltools_0.5.9
```